JOMO KENYATTA UNIVERSITY OF AGRICULTURE & TECHNOLOGY

SCHOOL OF OPEN, DISTANCE AND eLEARNING
P.O. Box 62000, 00200
Nairobi, Kenya
E-mail: elearning@jkuat.ac.ke

ICS 2104 Object-Oriented Programming I

◀◀ ▶▶
◀ ▶
◀ Doc Doc ▶
Back Close

This presentation is intended to covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green Exercise tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elearning@jkuat.ac.ke.

LESSON 5

Constructors and Destructors

Learning outcomes

Upon completing this topic you should be able to:

- Differentiate constructor and destructor
- List characteristics of constructor and destructor
- use constructors and destructors

5.1. Constructors

A constructor is a member function that is used for creating objects and initializing the states of the objects.

A constructor has the following characteristics:

- Its is the same as the name of the class within which it has been declared
- It is called automatically
- It does not have return type
- It is always declared within the public section of a class
- It is not inherited vi. A constructor can not be made virtual
- The address of the memory location occupied by a constructor can not be referenced

5.2. Declaration of Constructors

A constructor is declared and defined as follows:

```
class Rectangle
```

```
{
private:
int length;
int width;
. . .
public:
Rectangle(int l, int w); //declaration of constructor
. . .
}; //definition of constructor
Rectangle::Rectangle(int l, int w)
{
length=l;
width=w;
}
```

5.3. Calling Constructors

Since a constructor is always declared within the public section of a class, it is accessible from external functions(these are functions that are not members of the class that has the class).

For example the main method can call the constructor in the Rectangle class above as shown below.

Rectangle rect1(6, 5);

This statement is a call to Rectangle constructor in a class called Rectangle. The constructor is also passed two integer values, i.e. 6 and 5. The constructor will construct the object and 6 and 5 values will be used as the initial state of the object. The object that will be created will be referenced by rect1 variable. If we now want to pass a message to the object that has been created in Rectangle class, we have to use rect1 to refer to the

object as shown below rect1.find_area(); rect1 is our reference to the object created and find_area() is the message that we are passing to our object. rect1 refers to the object created.

5.4. Types of Constructors

There are three types of constructors:

- Default constructors
- Copy constructors
- Parameterized constructors

5.4.1. A Default Constructor

A default constructor is a constructor that is not passed any arguments when it is called. A default constructor can be created automatically by the compiler. For example, the default

constructor for Rectangle class above is :

```
class Rectangle
{
private: . . .
public:
Rectangle();
. . .
}; //definition of default constructor
Rectangle::Rectangle()
{
}
```

Note that the body of this constructor is empty. Now if we have a statement shown below within the main method

```
int main(void) {
```

```
Rectangle rect1;
. . .
return 0;
}
```

Statement Rectangle rect1; in the above main method is a call to the default. In that case if the Rectangle class do not have the default constructor the compiler will create one automatically.

5.4.2. Parameterized constructors

Parameterized constructors are constructors that are passed arguments when they are called. Unlike default constructors, parameterized constructors are not created automatically by the compiler if they are not defined in a class. And these construc-

tors must be defined. For example:

```
class Rectangle
{
private:
int length;
int width;
. . .
public:
Rectangle(int l, int w);
. . .
}; //definition of a parameterized constructor Rectangle::Rectangle(int l, int w) { length=l; width=w; }
```

There are two ways of passing arguments to parameteized constructors when they are called: by calling the constructor

explicitly and by calling the constructor implicitly. For example Rectangle rect1= Rectangle(6, 5); //explicit call Rectangle rect1(6, 5); //implicit call Implicit call is the one that is normally used.

5.4.3. Copy constructors

Copy constructors are constructors that used to create and initialize an object using another object. For example Rectangle rect1(6, 5); //a call to a parameterized constructor Rectangle rect2(rect1); // a call to a copy constructor to create and initialized rect2 object //using rect1 object

5.5. Destructors

A destructor is a member function that is used to free a location that is held by an object for use by another object.

An object is deleted when the flow of control leaves the scope within which it has been created.

A destructor has the following properties

- Its name is the same as the name of the class
- Its name is preceded by a tilde (~) character
- It is not passed arguments and it does not have return type not even void
- It is called implicitly
- A class can have only one destructor
- A destructor can not be overloaded

For example

```
Rectangle::~Rectangle()
{
cout<<"An object has been destroyed "<<endl;
}
```

Note that just like other member functions, constructors can have default arguments and they can be overloaded.

Revision Questions

EXERCISE 1. ✍ Differentiate a constructor and a destructor

Example ✎. List any FOUR characteristics of a destructor

Solution:

a) It is not passed any arguments

b) Its name is the same as a class and it is preceded by tilde (~) symbol.

c) It is called automatically

d) It does not have return types

□

EXERCISE 2. ✍ Explain why a constructor is public and has no return type

EXERCISE 3. ✍ Differentiate a default constructor, a copy constructor and parameterized constructor

EXERCISE 4. ✍ Explain the difference between the following two statement:

Car myCar(200);

Car yourCar;

References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

◀◀ ▶▶ ◀ ▶ ◀ Doc Doc ▶ Back Close

Solutions to Exercises

Exercise 1. A constructor creates objects while a destructor destroy objects

Exercise 1